RideLink — TVDE Ride-hailing Platform

ADMBD Database Administration — Project Report, Iteration 1

Group XX — [student number], [student number], [student number], [student number]

November 2026

Contents

1 Project Description and Requirements 2
1.1 Domain and motivation . . . 2
1.2 Actors and use cases . . . 2
1.3 Functional requirements . . . 2
1.4 Data requirements . . . 3
1.5 Typical database operations and workloads . . . 3

2 Selection of Database Technologies 4
2.1 Chosen DBMSs . . . 4
2.1.1 Why PostgreSQL . . . 4
2.1.2 Why MongoDB . . . 5
2.1.3 Why two systems (polyglot persistence) . . . 5
2.2 Alternatives considered . . . 5

3 Database Models Specification 6
3.1 Relational model (PostgreSQL) . . . 6
3.2 Document model (MongoDB) . . . 7
3.3 Data ownership — which DBMS manages what . . . 8
3.4 Cross-database consistency . . . 9

4 Deployment and Architecture Analysis 9
4.1 Initial architecture . . . 9
4.2 On-premises vs. cloud . . . 10
4.3 Chosen deployment approach . . . 10
4.4 Distribution, replication, availability and infrastructure requirements . . . 10
4.5 Prototype validation (performed in Iteration 1) . . . 11

5 Backup Policy Specification 12
5.1 Recovery objectives . . . 12
5.2 Strategy and schedule . . . 12
5.3 Recovery outline . . . 13

6 Scalability and Elasticity Plan 13
6.1 Anticipated needs . . . 13
6.2 PostgreSQL . . . 14
6.3 MongoDB . . . 14
6.4 Elasticity . . . 14
6.5 Validation plan for Iteration 2 . . . 15

7 Project Repository 15

8 Use of Generative AI 15

Group Self-Evaluation 16

Repository: https://github.com/TODO-org/ADMBD-groupXX — tag `iteration1` Teaching staff access: granted to `psm@isep.ipp.pt` and `ral@isep.ipp.pt` institutional GitHub accounts (see §7).

1 Project Description and Requirements

1.1 Domain and motivation

RideLink is a ride-hailing platform for the Portuguese TVDE market (Transporte em Veículo Descaracterizado a partir de Plataforma Eletrónica, Law 45/2018). Riders request a trip from the app, the platform matches the nearest suitable driver, tracks the trip in real time, charges the rider and settles the driver's earnings.

The domain was chosen because it naturally mixes two very different data workloads, which is what makes a polyglot design justifiable rather than artificial:

- Transactional, relational, regulated data — users, certified drivers and operators, vehicles, trips, payments, VAT invoices. It needs ACID guarantees, strong constraints and long fiscal retention (invoices must be kept for years).
- High-volume, geospatial, semi-structured data — GPS pings every few seconds, live driver positions, routes, chat, reviews, notifications. It is write-heavy, schema-flexible and mostly ephemeral.

The TVDE context adds concrete administration concerns: personal data under GDPR (location is personal data), mandatory driver documents with expiry dates, fiscal compliance, and strong peak/off-peak load variation (Friday/Saturday nights, airport, events).

1.2 Actors and use cases

Actor	Main use cases
Rider	Register; request a trip (pickup/dropoff, category, promo code); see price estimate; follow driver live; pay; rate driver; view history and invoices; chat with driver
Driver	Register and upload documents; go online/offline; receive and accept requests; navigate; complete trip; rate rider; view earnings and payouts
TVDE operator	Manage affiliated drivers and vehicles; view operator-level earnings
Administrator	Approve/suspend drivers; manage categories, tariffs and promo codes; monitor KPIs
Support agent	Handle tickets and incidents; review trip route and messages; issue refunds
DBA (system actor)	Backup/restore, monitoring, tuning, security, scaling

1.3 Functional requirements

ID	Requirement
FR1	Users register/authenticate with a role (rider, driver, admin, support).
FR2	Driver onboarding: TVDE certificate, documents with validity; approval workflow; automatic suspension when a mandatory document expires.

ID	Requirement
FR3	A driver is associated with one operator and one vehicle at a time; history of vehicle assignments is kept.
FR4	Riders request trips; the system finds available drivers of the requested category near the pickup point and assigns exactly one.
FR5	Trip life-cycle: requested → accepted → arrived → in progress → completed (or cancelled / no driver).
FR6	Live tracking of driver position during the trip; route stored on completion.
FR7	Fare computed from category tariff (base + km + minute), surge multiplier and promo code.
FR8	Payments by card, MB WAY or cash; VAT invoice issued per completed trip; platform fee and driver payout calculated.
FR9	Weekly driver payout statements.
FR10	Mutual rating and review after each trip.
FR11	In-trip chat and push notifications.
FR12	Support tickets and incident reports with attachments.
FR13	Reporting: trips per zone/hour, revenue, driver utilisation, cancellation rate.

1.4 Data requirements

- Integrity: a trip belongs to exactly one rider and (once accepted) one driver; a vehicle has one driver at a time; payments never exceed trip amount; invoice numbers are sequential and unique per series.
- Consistency: assignment of a driver and payment capture must be atomic (no double-assignment, no double-charge).
- Geospatial: point queries (nearest drivers), distance, zone aggregation.
- Retention/legal: fiscal data multi-year; GPS telemetry limited to what is needed (GDPR data minimisation) — 90 days raw, then aggregated; chat 30 days.
- Security: passwords hashed, NIF/IBAN/phone protected, role-based access (see Iteration 2).

1.5 Typical database operations and workloads

Sizing assumptions (mid-size city, Porto-like; to be validated and adjusted in Iteration 2 via benchmarks):

Parameter	Assumption
Registered riders / drivers	300 000 / 8 000
Trips per day	25 000 average; peak ≈ 3× the average hour
Active drivers at peak	≈ 1 500
GPS ping period	1 per 4 s per active driver

Derived: peak trip creation ≈ 2–3 trips/s; GPS writes at peak ≈ 1 500 / 4 s ≈ 375 writes/s; ≈ 7 M pings/day at ≈ 150–200 B each ≈ 1.2 GB/day ≈ 100 GB for the 90-day window (before compression). Trips: 25 000/day × ≈ 1 KB ≈ 9 M rows/year in `trip`, with payments/invoices of the same order.

#	Operation	Type	DBMS	Frequency
O1	Insert trip request	OLTP write	PostgreSQL	~25 k/day, 2–3/s peak

#	Operation	Type	DBMS	Frequency
O2	Assign driver to trip (transaction, row lock)	OLTP write	PostgreSQL	per trip
O3	Update trip status	OLTP write	PostgreSQL	~5 per trip
O4	Capture payment + issue invoice (transaction)	OLTP write	PostgreSQL	per trip
O5	Rider/driver trip history (paginated)	OLTP read	PostgreSQL	very frequent
O6	Daily/weekly revenue and payouts by driver/operator	Analytical	PostgreSQL replica	batch
O7	Documents about to expire	Batch read	PostgreSQL	daily
O8	Ingest GPS ping	High-rate write	MongoDB	~375/s peak
O9	Upsert live driver position	High-rate write	MongoDB	~375/s peak
O10	Nearest k available drivers within radius	Geo read	MongoDB	per trip request, 2–3/s peak
O11	Insert/read reviews and compute average	Mixed	MongoDB (avg cached in PG)	per trip
O12	Chat messages, notifications	Write/read	MongoDB	per trip
O13	Trip route lookup for support	Read	MongoDB	rare

The read/write ratio is write-heavy for telemetry (≈ 95 % writes) and read-heavy for history/reporting (≈ 90 % reads).

2 Selection of Database Technologies

2.1 Chosen DBMSs

Role	DBMS	Version
Relational (SQL)	PostgreSQL + PostGIS	16
Non-relational (NoSQL, document)	MongoDB	7

2.1.1 Why PostgreSQL

- Full ACID with row-level locking and multiple isolation levels — needed for driver assignment (O2) and payment/invoice atomicity (O4).
- Rich constraints: `CHECK`, `EXCLUDE` with range types (a vehicle/driver cannot have overlapping assignments), partial and expression indexes, enums, generated identity.
- Declarative partitioning (by month on `trip`) to keep indexes small and make retention/archival cheap.
- PostGIS for geospatial support in the relational side (zone analysis, distance in reports).
- Mature HA/backup ecosystem (streaming replication, WAL archiving, point-in-time recovery, pgBackRest) which maps directly to ADMBD topics; open source, no licence cost.

2.1.2 Why MongoDB

- Telemetry, reviews, chat and tickets have evolving, variable structure; documents avoid schema migrations and joins.
- Time-series collections (native since 5.0) with automatic bucketing and TTL expiry fit GPS data (O8) and GDPR-driven retention.
- `2dsphere` indexes with `$nearSphere` support the "nearest driver" query (O10) directly.
- Built-in replica sets (automatic failover) and sharding give a clear horizontal scaling path for the write-heavy part.

2.1.3 Why two systems (polyglot persistence)

Putting GPS at 375 writes/s and fiscal invoices in the same engine would couple very different requirements (durability and constraints vs. throughput and expiry). Separating them lets each be tuned, backed up and scaled according to its own needs. The cost — cross-system consistency — is controlled by keeping PostgreSQL as the system of record and referencing it from MongoDB by `tripId`/`driverId` only (no cross-DB transactions; see §3.4).

2.2 Alternatives considered

Category	Alternative	Why not chosen
SQL	MySQL/MariaDB	Weaker geospatial and constraint features (no range `EXCLUDE`, no partial indexes); PostGIS is the de-facto standard for geospatial SQL.
SQL	Oracle / SQL Server	Strong features, but licence cost and heavier footprint for an academic/start-up context; no functional gain for this scope.
SQL	CockroachDB / YugabyteDB	Excellent for global horizontal scale, but our scope is a single city/region; adds complexity and diverges from the course's mainstream tooling.
NoSQL	Redis	Ideal for the live driver position cache (`GEOSEARCH`) but limited persistence/query capability for the remaining collections. Kept as a possible future cache in front of O9/O10.
NoSQL	Cassandra	Very high write scalability, but poor ad-hoc/geospatial querying and heavier operations for a medium-size workload.
NoSQL	TimescaleDB (PG extension)	Strong for telemetry and keeps one engine; however it would not give the project a distinct NoSQL technology as required, and reviews/chat/tickets are document-shaped.
NoSQL	Neo4j	Graph model is not central here (no deep relationship traversal use case).

Category	Alternative	Why not chosen
NoSQL	DynamoDB	Locks the project into AWS and cannot be run locally for the prototype.

3 Database Models Specification

3.1 Relational model (PostgreSQL)

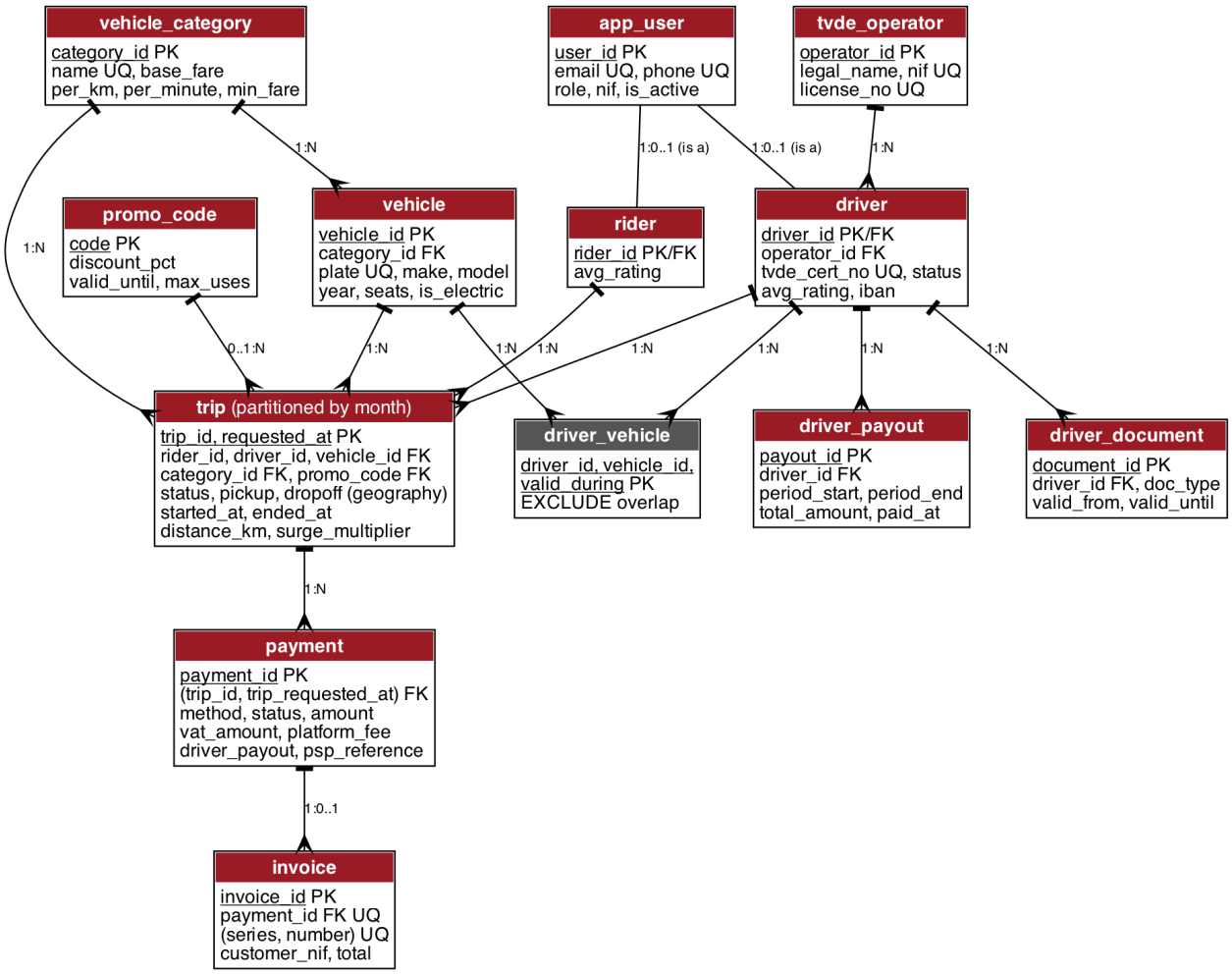


Figure 1: Relational ER diagram (generated from `docs/diagrams/er.dot`)

The full DDL is in `sql/01_schema.sql`. Main entities:

Entity	Identifier	Key attributes	Main constraints
`app_user`	`user_id`	email, phone, password_hash, role, nif	`email`, `phone` UNIQUE; email format `CHECK`; NIF = 9 digits
`driver` (specialises user)	`driver_id` (FK to `app_user`)	tvde_cert_no, status, iban, operator_id	cert UNIQUE; status enum

Entity	Identifier	Key attributes	Main constraints
rider (specialises user)	rider_id (FK)	avg_rating	rating 1–5
tvde_operator	operator_id	legal_name, nif, license_no	NIF and licence UNIQUE
driver_document	document_id	doc_type, valid_from, valid_until	valid_until > valid_from; UNIQUE per driver/type/start
vehicle_category	category_id	name, base_fare, per_km, per_minute, min_fare	tariffs ≥ 0
vehicle	vehicle_id	plate, make, model, year, seats	plate UNIQUE; seat/year ranges
driver_vehicle	composite (3 cols)	time range	EXCLUDE USING gist: no overlapping use of a car or by a driver
trip	(trip_id,requested_at)	rider, driver, vehicle, category, status, pickup/dropoff geography, times, distance, surge	monthly range-partitioned; ended_at >= started_at; a driver is mandatory from ACCEPTED on
promo_code	code	discount_pct, valid_until, max_uses	pct 0–100
payment	payment_id	method, status, amount, VAT, fee, payout	amounts ≥ 0; FK to trip
invoice	invoice_id	series, number, total	(series,number) UNIQUE; 1:1 with payment
driver_payout	payout_id	period, total, paid_at	UNIQUE per driver/period

Relationships: app_user 1–0..1 driver/rider (specialisation); tvde_operator 1–N driver; driver N–M vehicle via driver_vehicle (temporal); vehicle_category 1–N vehicle and trip; rider/driver/vehicle 1–N trip; trip 1–N payment (retries or split payment) → 1–0..1 invoice.

Design decisions worth noting

- trip is partitioned by requested_at, so the primary key and foreign keys from payment include that column (a PostgreSQL requirement). This is a deliberate trade-off: cheaper retention and smaller indexes versus a composite key.
- Monetary values use NUMERIC(10,2), never floating point.
- Role specialisation (driver, rider) is kept in separate tables to hold role-specific columns without nulls; the role column in app_user supports authentication/authorisation.
- The 3NF is respected; the only intentional denormalisation is avg_rating on driver/rider, a materialised value refreshed from MongoDB reviews to avoid cross-DB calls in dispatch ranking.

3.2 Document model (MongoDB)

Collections and example documents (full definitions, validators and indexes in mongo/01_collections.js):

Collection	Purpose	Document shape (abridged)	Indexes / options
driver_locations	Raw GPS history	{ ts, meta:{driverId, tripId}, loc:{Point}, speedKmh, heading }	time-series (timeField: ts, metaField: meta); TTL 90 days; 2dsphere on loc
driver_live_state	Current position and availability, one doc per driver	{ _id: driverId, status, categoryId, loc, updatedAt }	2dsphere; compound (status,categoryId); TTL 300 s on updatedAt
trip_routes	Compact route per completed trip	{ tripId, polyline, distanceKm, events:[{type, ts, loc}] }	unique tripId; $jsonSchema validator
reviews	Ratings and comments	{ tripId, fromUserId, toUserId, stars, tags:[], comment, createdAt }	(toUserId,createdAt); unique (tripId,fromUserId); validator 1–5
trip_messages	In-trip chat	{ tripId, from, text, sentAt }	(tripId,sentAt); TTL 30 days
notifications	Push/in-app events	{ userId, type, payload{}, read, createdAt }	(userId,createdAt)
support_tickets	Incidents, attachments	{ userId, tripId?, category, status, thread:[{author,text,ts}], attachments:[] }	(userId,status)
audit_log	Security/audit trail	{ ts, actorId, action, target, ip }	ts, (actorId,ts)

Modelling choices: embedding is used where data is read together and bounded (events inside trip_routes, thread inside a ticket); referencing (by numeric tripId/userId from PostgreSQL) is used where documents grow without bound or are queried independently (GPS points, messages, reviews). No document may exceed the 16 MB BSON limit by design.

3.3 Data ownership — which DBMS manages what

Data	PostgreSQL	MongoDB	Rationale
Users, riders, drivers, operators, documents	•		Master data, constraints, GDPR erasure workflow
Vehicles, categories, tariffs, promo codes	•		Reference data with integrity
Trips (state, price, locations)	•		System of record, transactions
Payments, invoices, payouts	•		ACID, fiscal retention
Live driver position		•	Very high update rate, geo query, ephemeral
GPS history / routes		•	Time-series, TTL, volume
Reviews, chat, notifications, tickets, audit		•	Variable structure, high volume

Data	PostgreSQL	MongoDB	Rationale
Average rating (derived)	• (copy)	• (source)	Refreshed periodically; avoids cross-DB call in dispatch

3.4 Cross-database consistency

There are no distributed transactions. Rules: (1) PostgreSQL is the authority; MongoDB documents reference IDs only. (2) Writes that must be atomic (assignment, payment, invoice) happen only in PostgreSQL. (3) MongoDB writes are idempotent (`upsert`, unique indexes) so they can be retried. (4) A nightly reconciliation job detects orphan Mongo documents (e.g. `tripId` not in PostgreSQL).

4 Deployment and Architecture Analysis

4.1 Initial architecture

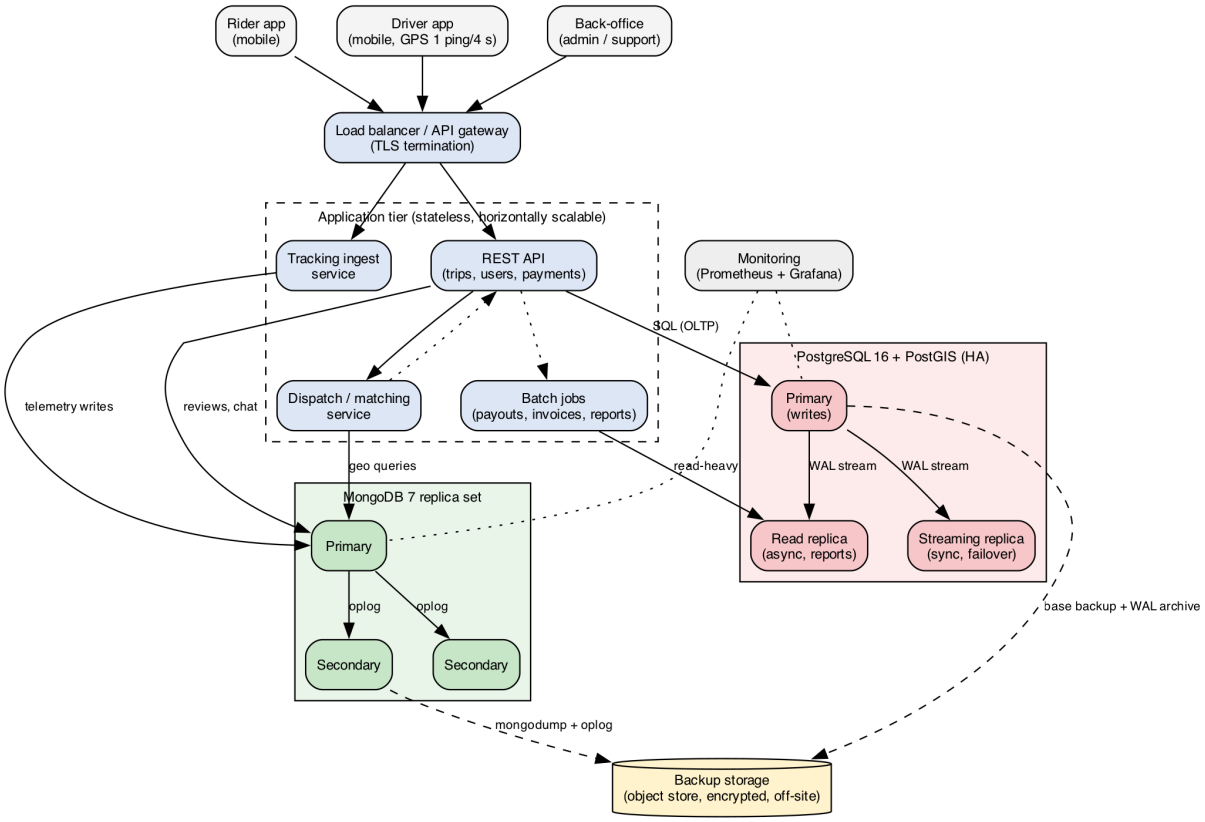


Figure 2: Initial architecture (generated from `docs/diagrams/architecture.dot`)

- Application tier is stateless (API, dispatch, tracking ingest, batch jobs) and sits behind a load balancer with TLS termination.
- PostgreSQL runs as one primary, one synchronous standby (failover, zero data loss) and one asynchronous read replica used by reports and batch jobs.
- MongoDB runs as a 3-member replica set (one primary, two secondaries) with majority write concern for critical documents and relaxed write concern for raw GPS.
- Backups go to encrypted, off-site object storage; monitoring covers both engines.

4.2 On-premises vs. cloud

Criterion	On-premises	Cloud (IaaS / managed)
Upfront cost	High (servers, networking, licences, space)	Low (pay-as-you-go)
Running cost	Predictable, cheaper at constant high utilisation	Higher at steady state, but follows demand
Elasticity	Limited by purchased hardware; weeks to scale	Minutes; fits strong night/weekend peaks
Availability / DR	Must build second site and network	Multi-AZ / multi-region built in
Operations	Full control; the team must do patching, backup, HA	Managed services (RDS/Cloud SQL, MongoDB Atlas) offload much of it, with less low-level control
Data protection / GDPR	Full physical control	Must choose EU regions, DPA with provider; provider lock-in considerations
Latency	Local	Region choice (e.g. Paris/Frankfurt/Madrid) is adequate for a single-country app
Skills	Requires specialised DBA/infrastructure staff	Reduced infrastructure effort

4.3 Chosen deployment approach

Cloud, EU region, with self-managed-style database layout on VMs/containers (IaaS or Kubernetes), multi-availability-zone. Justification:

1. Demand is spiky and unpredictable (events, weather, nights); elasticity avoids paying for peak capacity all month.
2. A start-up/academic context has no capital budget for hardware and no second data centre.
3. GDPR is satisfied by keeping all data in EU regions, encrypting at rest and in transit, and signing a DPA with the provider.
4. Multi-AZ gives the availability target (§4.4) without building physical DR.
5. We prefer open-source engines on IaaS (or managed equivalents of the same engines) to limit lock-in and keep the ability to move.

A managed service (e.g. managed PostgreSQL and MongoDB Atlas) is the natural production evolution; for the course prototype we will use Docker containers on a laptop/VM (PostgreSQL primary + replica, MongoDB 3-node replica set), which reproduces the same topology at small scale. This difference is intentional and documented.

4.4 Distribution, replication, availability and infrastructure requirements

Requirement	Target	How
Availability	99.9 % (≈ 8.7 h/year) for booking and trip flow	Multi-AZ, automatic failover (Patroni for PG, replica-set election for Mongo)
PostgreSQL replication	1 sync standby + 1 async replica	Streaming replication; synchronous for zero RPO on failover; async replica for read offloading

Requirement	Target	How
MongoDB replication	Replica set, 3 members, `w:majority` for reviews/tickets, `w:1` for GPS	Elections in ≈ 10–12 s; no single point of failure
Geographic distribution	Single region (Portugal-only service); off-site backup copy in a second region	Cross-region backup replication
Network / security	Private subnets for databases, TLS between all components, no public DB endpoints	Security groups; bastion for admin
Storage	PG ≈ 0.3 TB year 1; Mongo ≈ 150 GB (90-day telemetry with compression)	SSD volumes with growth alerts at 70 %
Monitoring	Latency, replication lag, connections, locks, disk, slow queries	Prometheus exporters + Grafana, alerts

4.5 Prototype validation (performed in Iteration 1)

The topology of §4.1 was reproduced at small scale with `docker compose up -d` (`docker-compose.yml`) and checked as follows. All credentials in the repository are test values for the local prototype only.

Check	Result observed
Schema scripts `01_schema.sql` + `02_partitions.sql` run on PostGIS 16	Executed without errors; monthly partitions created (previous month to +3 months)
Constraints	Overlapping `driver_vehicle` ranges rejected by `EXCLUDE`; trip without driver in `ACCEPTED` rejected by `CHECK`
Seed `03_seed.sql` (2 000 riders, 200 drivers, 20 000 trips, ≈ 18 000 payments/invoices)	Rows routed to the correct monthly partitions
Operations O1–O7 (`04_queries.sql`)	All execute, including the transactional assignment (O2) and payment + invoice (O4)
PostgreSQL streaming replication	Standby shows `streaming`, sync state `quorum` (`ANY 1 (replica1)`); the replica returns the same row counts as the primary (20 001 trips)
MongoDB replica set (3 nodes) + `01_collections.js`, `02_seed.js`, `03_queries.js`	Collections, validators and indexes created; 200 live positions, 20 000 pings, 2 000 reviews loaded; `$nearSphere` returns nearest drivers; validator rejects `stars = 9`
MongoDB failover	Primary stopped: a secondary was elected in < 20 s, data intact; the old primary rejoined as secondary

Lessons learned worth recording: (i) enabling synchronous replication from the first start blocks the bootstrap (commits wait for a standby that does not exist yet), so it is enabled in the last initialisation step; (ii) with synchronous replication the loss of the standby blocks writes on the primary unless the setting is relaxed (`ANY 1` among several standbys in production avoids this); (iii) rows already stored in the `DEFAULT` partition prevent creating the partition for their month, so partitions must exist before loading data. Backup tooling, PITR and sharding are not yet implemented — they are the core of Iteration 2.

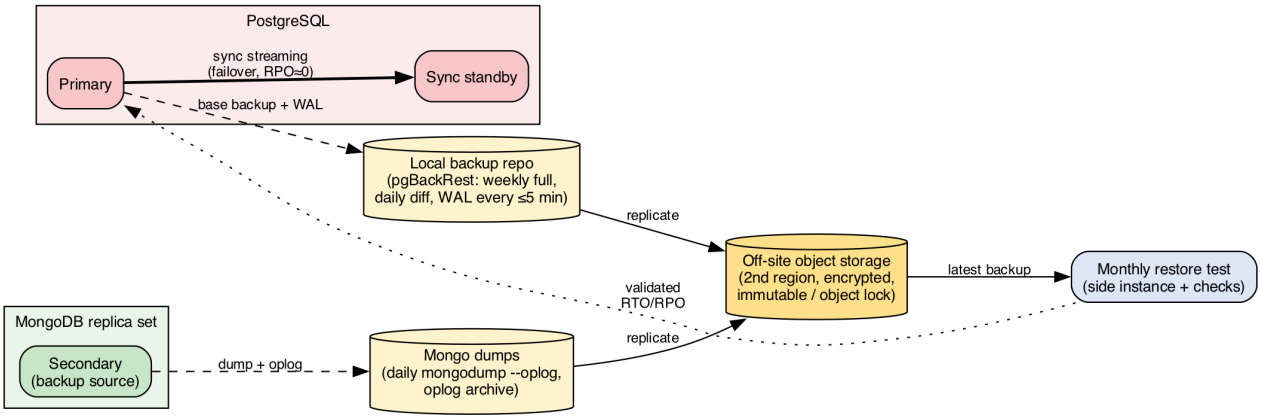


Figure 3: Backup and recovery flow (generated from `docs/diagrams/backup.dot`)

5 Backup Policy Specification

5.1 Recovery objectives

Objectives differ per data class because their business value differs:

Data set	RPO (max data loss)	RTO (max downtime)	Rationale
PostgreSQL (trips, payments, invoices)	≤ 5 min (≈ 0 after failover to sync standby)	≤ 1 h	Financial/fiscal data; cannot be lost
MongoDB operational (reviews, tickets, audit, notifications)	≤ 15 min	≤ 2 h	Important but not fiscal
MongoDB telemetry (`driver_locations`, `driver_live_state`)	up to 24 h acceptable	≤ 4 h (live state rebuilds itself in < 1 min as apps ping)	Ephemeral/derived; live state does not need restore

5.2 Strategy and schedule

	PostgreSQL	MongoDB
Tool	pgBackRest (physical full/diff/incr + WAL archiving); `pg_dump` for logical schema/data export	mongodump with `--oplog` (consistent snapshot) for the prototype; volume snapshots or Percona Backup for MongoDB in production
Full	Weekly (Sunday 02:00)	Daily (03:00) for operational collections
Differential / incremental	Daily differential (02:00)	Oplog continuously archived (point-in-time)
Continuous	WAL archiving every ≤ 5 min (`archive_timeout`) → PITR	Oplog window ≥ 24 h
Logical	Nightly `pg_dump -Fc` of schema + reference tables (portable, tests restores)	Monthly export of `reviews`/`support_tickets` as JSON for archival
Format	Compressed (zstd), encrypted (AES-256), checksummed	Gzip archive, encrypted

	PostgreSQL	MongoDB
Excluded	—	`driver_locations` and `driver_live_state` are not in the daily backup (TTL data, ephemeral); only aggregated daily summaries are archived

Retention

Backup	Retention
PG full backups	4 weekly + 12 monthly
PG WAL / PITR window	7 days
Invoices / payments (archive export)	Multi-year, per Portuguese fiscal retention rules (exact period to be confirmed against current legislation)
Mongo daily dumps	14 days, plus 6 monthly
Offsite copy	All of the above replicated to a second region, immutable (object lock)

GDPR note: backups also contain personal data; erasure requests are applied to live data and the backup expiry (≤ 12 months) is documented in the privacy policy.

5.3 Recovery outline

1. Failure of a single node → automatic failover (PG standby promoted / Mongo election); no restore (RTO minutes).
2. Logical error (wrong `DELETE`/`UPDATE`) → PITR with pgBackRest to a timestamp before the error into a side instance; extract rows and re-apply; for Mongo, restore the collection with `mongorestore --nsInclude` and replay the oplog to a chosen point.
3. Loss of the whole site/AZ → rebuild the cluster from IaC, restore latest full + differential + WAL from the second region, start the app tier, MongoDB live state repopulates from driver pings.
4. Verification: every restore is checked (row counts, `pg_checksums`, application smoke test) and a restore test is performed monthly (and in Iteration 2 demonstrated with measured RTO/RPO).

6 Scalability and Elasticity Plan

6.1 Anticipated needs

Dimension	Today (launch)	12 months	36 months
Trips/day	25 k	75 k	250 k
Peak GPS writes/s	≈ 375	≈ 1 100	≈ 3 700
`trip` rows (cumulative)	9 M	36 M	> 150 M
PG size	≈ 90 GB	≈ 350 GB	> 1.5 TB
Mongo raw GPS (90 d)	≈ 100 GB	≈ 300 GB	≈ 1 TB

(Assumptions in §1.5; figures are order-of-magnitude estimates to be validated by load tests in Iteration 2.) Elasticity matters because of daily and weekly peaks (≈ 3× average).

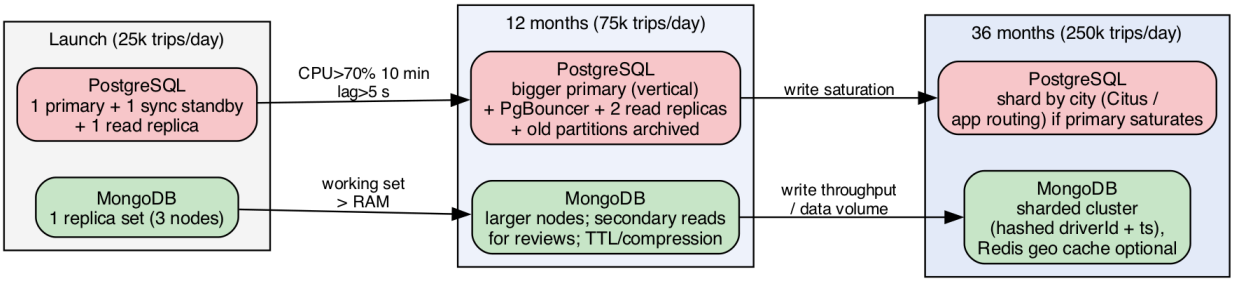


Figure 4: Scaling path per DBMS (generated from `docs/diagrams/scaling.dot`)

6.2 PostgreSQL

- Vertical first: more CPU/RAM/IOPS on the primary is the cheapest step; `shared_buffers`, `work_mem` and connection pooling (PgBouncer) tuned.
- Read scaling (horizontal): add asynchronous replicas for history, reports and batch jobs (O5, O6); route by workload at the application level.
- Data growth: monthly partitions of `trip` (and later `payment`) → detach/archive old partitions to cheap storage, keep hot partitions small and indexes cache-resident.
- Write scaling (later): if a single primary saturates (not expected before the 36-month column), shard by city/region with Citus or application-level routing; trips are naturally partitioned by region.
- Trade-offs: replicas introduce replication lag (reads may be stale — never use them for assignment or payment); sharding complicates cross-shard queries and the composite keys; vertical scaling has a ceiling and needs maintenance windows (mitigated by switchover to a standby).

6.3 MongoDB

- Write scaling (horizontal): the GPS workload is the main driver. Enable sharding on `driver_locations` with a hashed or compound key (e.g. `meta.driverId` + `ts`) to spread writes evenly and avoid a hot shard on time; keep `driver_live_state` sharded by `_id` hash.
- Read scaling: secondary reads (`readPreference: secondaryPreferred`) for reviews/history where slight staleness is acceptable; never for the nearest-driver query (O10), which needs fresh data.
- Vertical: RAM to keep working set (live state + indexes) in memory.
- Data growth: TTL indexes and time-series compression cap volume; tiered archival of aggregates.
- Optional cache: Redis `GEOSEARCH` in front of O9/O10 if latency targets are missed.
- Trade-offs: shard-key choice is hard to change; `$nearSphere` with sharding may become scatter-gather (mitigated by routing per city zone); majority writes cost latency; more shards = more operational cost.

6.4 Elasticity

- Application tier: auto-scale on CPU/requests (stateless) — the first line of defence for peaks.
- Database tier: replicas and shards are added ahead of known peaks (scheduled scaling, events) rather than reactively, because databases need minutes to hours to warm up and rebalance; scale-in is performed only off-peak.
- Triggers (to be refined in Iteration 2): CPU > 70 % for 10 min, replication lag > 5 s, p95 query latency above target, disk > 70 %.
- Cost trade-off: over-provisioning is simplest but wasteful; fully reactive scaling risks degraded service during ramp-up — a mix of scheduled and reactive is proposed.

6.5 Validation plan for Iteration 2

Load generator reproducing O1–O13 (e.g. k6/pgbench/custom script), measurements before and after: partitioning, index changes, read-replica offload, MongoDB sharding on a small test cluster. Results will be reported with evidence.

7 Project Repository

- URL: https://github.com/TODO-org/ADMBD-groupXX (to be filled in)
- Teaching staff access: `psm@isep.ipp.pt` and `ral@isep.ipp.pt` — invitation sent with their institutional GitHub accounts (confirm in the final version).
- Documentation format: Markdown (`docs/report.md`); PDF generated with `make pdf` (pandoc + XeLaTeX); diagrams from Graphviz sources in `docs/diagrams/`.
- Git tag: `iteration1`, pushed before the Moodle deadline.

Repository layout:

```
docs/              report.md, diagrams/*.dot, generated PDF
sql/               01_schema, 02_partitions, 03_seed, 04_queries (PostgreSQL)
mongo/             01_collections, 02_seed, 03_queries (MongoDB)
docker/            init scripts for replication and replica-set setup
docker-compose.yml  prototype topology (PG primary + sync standby, 3-node Mongo RS)
Makefile           build diagrams and PDF
```

8 Use of Generative AI

Tool	Purpose	Artefacts / tasks	Review and validation
Claude (Anthropic)	Brainstorming of the domain, drafting of the report structure and first versions of the schema and document model, diagram sources	Sections 1–6, `sql/*.sql`, `mongo/*.js`, `docker-compose.yml` and init scripts, `docs/diagrams/*.dot`	(the team must complete this column truthfully): e.g. each section reviewed and adapted by the author named in the self-evaluation; SQL tested on a PostgreSQL 16 instance and JavaScript in `mongosh`; technical claims (partitioning rules, time-series collections, TTL, pgBackRest) checked against official documentation; workload numbers recalculated by the team

(If the team changes its use of GenAI, this section must be updated to reflect what was actually done.)

Group Self-Evaluation

Student number	Did not work	Far be-low	Slightly below	Average	Slightly above	Far above	Did all the work	Tasks the student was responsible for
20XXXXX1				X				Project description, DB technologies
20XXXXX2				X				Database models
20XXXXX3				X				Deployment and architecture, scalability
20XXXXX4				X				Backup policy, repository, GenAI section

(Placeholders: replace by the real numbers, ratings and tasks.)